

Sephora Reviews — Data Engineering Capstone

Project Plan · Postgres 3NF OLTP → Star-Schema Warehouse → Airflow → Power BI

Revision 2 · 6 August 2026 · for review before implementation begins

Changed in revision 2. The warehouse schema now builds on my earlier hand-written schema (`reference/02_warehouse_schema.sql`) rather than replacing it — its integer `YYYYMMDD` date key, data-driven `dim_date` range, FK indexes, CHECK constraints and `dim_brand` are all kept (D12). Two corrections were made to it: `dim_customer` is split into identity plus a junk dimension after measuring that 13.69% of reviews would otherwise be mis-tagged (D2), and `brand_key` is dropped from the fact table (D11). Separately, the `highlights` column was evaluated and **deliberately descoped**, taking business question #5 with it — the project now answers four questions and has no bridge tables (D3).

1,094,411 REVIEWS	8,494 PRODUCTS	503,216 AUTHORS	304 BRANDS	2008–2023 DATE SPAN
----------------------	-------------------	--------------------	---------------	------------------------

1. Context and objective

This is a capstone for a data engineering course, presented in 8 minutes. A classmate's finished project — a Synthea EHR warehouse (Postgres OLTP → star schema → Airflow → Power BI) — defines the exact format taught in the course. This plan reproduces that structure file-for-file: repo layout, module boundaries, code style, SQL migration conventions, and documentation set — applied to Sephora product and review data instead.

The presentation must cover five points, and each maps to a concrete artifact in the repo:

Presentation point	Artifact that answers it
1. Introduction to the data source	<code>docs/problem statement and data sources.md</code> + <code>explore.py</code> output
2. Data flow architecture	README mermaid diagram + ERD
3. Design decisions	<code>docs/09_decision_log.md</code> (D1–D10 below)
4. Airflow DAG (full + incremental)	<code>dags/sephora_dw_pipeline_staged.py</code> + graph screenshot
5. Data visualization	Power BI, 2 pages + <code>sql/analytics/</code> views

2. Data source — profiled, not assumed

Kaggle *Sephora Products and Skincare Reviews*. Already present in `data/raw/`. Every figure below was measured from the actual files, not taken from the dataset description.

product_info.csv — 8,494 rows × 27 columns, grain = one product

- `product_id` unique, **0 duplicates**. `brand_id` → `brand_name` is strictly 1:1 across 304 brands.
- Nulls: `rating` / `reviews` 278 · `size` 1,631 · `variation_type` 1,444 · `ingredients` 945 · `highlights` 2,207 · `tertiary_category` 990 · `secondary_category` 8 · `value_price_usd` 8,043 · `sale_price_usd` 8,224 · `child_max/min_price` 5,740.
- Categories: 9 primary (Skincare 2,420 · Makeup 2,369 · Hair 1,464 · Fragrance 1,432 · Bath & Body 405 · Mini Size 288 · Men 60 · Tools & Brushes 52 · Gifts 4), 41 secondary, 118 tertiary — **174 distinct (primary, secondary, tertiary) triples**.

- **The hierarchy is not clean:** one secondary category appears under up to **7** different primaries. This drives decision D1.
- **highlights** is a stringified Python list → **112 distinct tags**, ~4.8 per product, covering 82.4% of reviewed products. A genuine many-to-many — **evaluated and deliberately descoped**, see D3.
- Price sanity: no `price_usd <= 0`, no `sale_price > price`, no rating outside 1–5.

reviews_*.csv — 5 files, 1,094,411 rows × 18 columns, grain = one review

- 503,216 distinct authors; 2,351 distinct products reviewed.
- **submission_time** spans **2008-08-28** → **2023-03-21**, 0 unparseable dates. By year: 2018 98K · 2019 144K · 2020 215K · 2021 202K · 2022 192K · **2023 49,531**.
- **Referential integrity verified: 0 orphan reviews** — every `product_id` exists in `product_info.csv`.
- **Duplicates:** 1,040 on `(author_id, product_id, submission_time)`; 5,525 on `(author_id, product_id)` alone. The gap is legitimate re-reviews on different dates — see D4.
- Nulls: `helpfulness` 561,592 · `hair_color` 226,768 · `eye_color` 209,628 · `skin_tone` 170,539 · `is_recommended` 167,988 · `skin_type` 111,557 · `review_title` 310,654 · `review_text` 1,444.
- `helpfulness` is null **exactly** when `total_feedback_count = 0` — verified on every row, as is `pos + neg = total`. It is derivable, not missing — see D5.
- `product_name` / `brand_name` / `price_usd` are duplicated onto every review row and **agree 100%** with `product_info.csv` — redundant, dropped at the staging boundary.
- Rating skew: 5★ 698,951 · 4★ 199,389 · 3★ 81,816 · 2★ 53,032 · 1★ 61,223.
- Reviewer attributes vary **per review**, not per author: **22,503 authors (4.47%)** give more than one distinct value, affecting **149,788 reviews (13.69%)**. Only **2,003** distinct four-attribute combinations actually occur. This drives D2.
- **0 rows have a time component** on `submission_time` — date grain is sufficient throughout.
- Two casing/sentinel defects found: `eye_color` holds both `'Grey'` and `'gray'`, and `skin_tone` holds `'notSureST'` — both normalised in `clean.py`.

3. Data flow architecture

```
data/raw/*.csv
| explore.py      profiling / inconsistency report
| clean.py        → data/processed/*.csv
▼
sephora_oltp — raw schema      1:1 CSV mirror, loaded by ingest.py (COPY)
|           3nf schema        9 normalized tables, FKs enforced
|           staging schema     trimmed, analytics-ready subset
▼ etl/ package: extract → transform → quality gate → load
sephora_dw   dw schema — 5 dimensions + fact_reviews
▼
Power BI (2 pages, live connection)
```

Orchestrated two ways, exactly like the reference: `pipeline.py` for local runs and `dags/sephora_dw_pipeline_staged.py` (Airflow 3.3.0 in Docker) for scheduled runs.

4. OLTP — 3NF design (`sephora_oltp`)

The reference's OLTP was a source mirror only. This project's goal #3 explicitly requires a 3NF database, so a normalized layer sits between raw and staging. Three layers, one purpose each.

raw — 1:1 with the two source shapes, all columns, type-tolerant, no cleaning. `raw.product_info` and `raw.reviews` (with a `source_file` column so every row traces back to its CSV).

3nf — normalized, foreign keys enforced:

Table	Key	Notes
<code>brand</code>	<code>brand_id</code> PK	304 rows; <code>brand_name</code> UNIQUE (1:1 verified)
<code>category</code>	<code>category_id</code> serial PK	174 rows; UNIQUE on (primary, secondary, tertiary)
<code>product</code>	<code>product_id</code> PK	FK → brand, FK → category; price, flags, size, variation, loves_count
<code>author</code>	<code>author_id</code> PK	503,216 rows; no descriptive attributes — see D2
<code>skin_tone</code> , <code>skin_type</code> , <code>eye_color</code> , <code>hair_color</code>	serial PK each	14 / 4 / 6 / 7 rows; lookup tables
<code>review</code>	<code>review_id</code> PK	FK → author, product, and the 4 lookups; rating, is_recommended, feedback counts, submission_date, title, text

`review_id` is **deterministic**: `md5(author_id || product_id || submission_time || row_ordinal)` . Re-running ingestion produces identical keys, which is what makes `ON CONFLICT DO NOTHING` work at the OLTP layer as well as the warehouse layer.

staging — the analytics-ready trim the ETL package actually reads: `staging.product` (brand name and the three category columns flattened in), `staging.review` (no review text, `review_length` precomputed), Built by `INSERT... SELECT` migrations, exactly like the reference's staging loads.

5. Data warehouse — star schema (`sephora_dw` , schema `dw`)

Grain: one row per review.

This schema keeps the parts of my earlier hand-written schema ([reference/02_warehouse_schema.sql](#)) that were already right — the `YYYYMMDD` integer date key, the data-driven `dim_date` range, FK indexes, CHECK constraints and `IF NOT EXISTS` DDL — and corrects one measured defect in `dim_customer` (D2).

Table	Key	Contents
<code>dim_date</code>	<code>date_key</code> INT (YYYYMMDD)	<code>full_date</code> UNIQUE; year, month, month_name, quarter, day_of_week, day_name, is_weekend. Range derived from the data with 30 days padding, <code>ON CONFLICT DO NOTHING</code> so it extends safely over time
<code>dim_brand</code>	<code>brand_key</code> serial	<code>brand_id</code> UNIQUE, brand_name — 304 rows
<code>dim_product</code>	<code>product_key</code> serial	<code>product_id</code> UNIQUE, product_name, FK → <code>dim_brand</code> , all three category levels flattened in (the star-vs-3NF contrast to show in the presentation), price_usd, price_band, size, loves_count, limited_edition, new, online_only, sephora_exclusive, out_of_stock
<code>dim_customer</code>	<code>customer_key</code> serial	<code>customer_id</code> UNIQUE — shopper identity only , no descriptive attributes (D2)
<code>dim_reviewer_profile</code>	<code>reviewer_profile_key</code> serial	Junk dimension, 2,003 rows : distinct (skin_tone, skin_type, eye_color, hair_color) combinations, nulls → <code>'Unknown'</code> , UNIQUE on all four
<code>fact_reviews</code>	<code>review_key</code> serial	<code>UNIQUE(source_row_id, product_id)</code> drives <code>ON CONFLICT</code> ; FKs to product / customer / reviewer_profile / date; measures: rating (<code>CHECK BETWEEN 1 AND 5</code>), is_recommended, helpfulness, total_feedback_count, total_pos_feedback_count, total_neg_feedback_count, review_length; <code>submission_date</code> retained for the watermark

Indexes on all four fact FK columns. **No `brand_key` on the fact table** — see D11.

6. Design decisions to defend

D1 — The category "hierarchy" isn't one, so it isn't modeled as one. 41 secondary categories map to up to 7 different primaries. Modeling primary → secondary → tertiary as nested levels would assert a structure the data doesn't have. `category` is keyed on the full triple (174 rows); the star schema flattens all three onto `dim_product`.

D2 — Reviewer attributes belong to the review, not the author. My earlier schema modelled these as attributes of `dim_customer`, keyed UNIQUE on the customer. Measured against the data, that doesn't hold: **22,503 authors (4.47%)** give more than one distinct value — 12,525 for `skin_tone`, 8,387 for `skin_type`, 7,614 for `hair_color`, 827 for `eye_color`. Those authors wrote **149,788 reviews (13.69% of the dataset)**. One row per customer forces one profile per person, so roughly **1 review in 7 would carry the wrong reviewer profile** — silently, with no error — and it would break the exact business question the table exists to answer. Fixed with a **junk dimension**: `dim_reviewer_profile` holds one row per distinct four-attribute combination (**2,003 of a possible 4,200**), each review points at the combination it actually recorded, and `dim_customer` keeps identity only. SCD Type 2 was rejected — the source has no reliable change dates; putting four text columns on the fact table was rejected — 1M rows of repeated strings.

D3 — highlights evaluated, then deliberately dropped. The column holds a stringified list of marketing tags — 112 distinct values, ~4.8 per product, covering 82.4% of reviewed products and 89.6% of all reviews. It is the dataset's one genuine many-to-many, and storing it would require `highlight` + `product_highlight` in the 3NF layer and a `dim_highlight` + bridge in the warehouse — because a cell holding `['Vegan', 'Hydrating', 'Cruelty-Free']` breaks 1NF, and keeping the raw list in a text column inside a database claiming 3NF was never an option. Weighed against an 8-minute presentation and the many-to-many filter complexity it introduces in Power BI, the column is **cut at the cleaning step** and business question #5 dropped with it. `explore.py` still reports the tag distribution, so the decision is visibly informed rather than accidental — the same pattern the reference project used when it ingested `claims_transactions.csv`, verified its join, then descope'd it.

D4 — Deduplication key is (author_id, product_id, submission_time). Measured: 1,040 duplicates on the triple versus 5,525 on the (author, product) pair. The ~4,500-row difference is legitimate re-reviews written on different dates — deduplicating on the pair would silently delete real data. Both numbers go in the decision log so the choice is auditable.

D5 — helpfulness nulls are not imputed. Verified that `helpfulness IS NULL` $\Leftrightarrow$ `total_feedback_count = 0` on every row. The value is undefined, not missing. Filling it with 0 would assert "everyone found this unhelpful," a different and false claim. It stays NULL, and every helpfulness measure filters on `total_feedback_count > 0`.

D6 — Review text stops at the OLTP boundary. Full `review_text` and `review_title` live in `3nf.review` for traceability; `fact_reviews` carries `review_length` instead. Roughly 500 MB of free text on the fact table would slow every load and every query to answer a question no KPI asks. Same discipline as the reference's rule of storing each measure at the grain its question needs.

D7 — Two physical databases, `sephora_oltp` and `sephora_dw`. OLTP-shaped and OLAP-shaped data are conceptually different systems, and the split makes the pipeline boundary concrete and demoable. Trade-off accepted: moving data requires Python with two connections rather than a single cross-schema `INSERT...SELECT`.

D8 — Incremental split: full load = reviews through 2022-12-31, incremental = 2023-01-01 onward. 49,531 real rows held back across three months (Jan 16,907 · Feb 16,754 · Mar 15,870) — enough to show the watermark advancing across multiple runs using real data, with no synthetic row generation. Mirrors the reference's monthly-batch approach.

D9 — Surrogate keys come from Postgres `serial`, never from pandas. Dimensions load keyless in parallel and the keys are read back afterwards via a lookup query, so no auto-increment state has to be coordinated

across parallel branches.

D10 — Products and brands load in full every run; only reviews are incremental. Dimensions have no time axis. Loading them in full means a review arriving for a newly-listed product is never silently dropped by `_drop_unmatched` — the exact bug the reference project hit and fixed with its `extract*_all` functions.

D11 — `brand_key` removed from the fact table. My earlier schema carried `brand_key` on both `fact_reviews` and `dim_product`. Every product has exactly one brand, so the fact copy adds no information while creating a way for the two to disagree after a bad load. Brand lives on `dim_product` only; brand analysis joins through it.

D12 — `dim_date` keeps the integer `YYYYMMDD` key and the data-driven range. Both carried over unchanged from my earlier schema, which got this right: the integer key is the standard Kimball convention, and deriving the range from `MIN/MAX(submission_time)` with 30 days of padding (plus `ON CONFLICT DO NOTHING`) means the dimension can't go stale when new data arrives, unlike a hardcoded range. Verified safe: **0 of 1,094,411 rows** carry a time component, so day grain loses nothing.

D13 — `UNIQUE(source_row_id, product_id)` kept as the idempotency key. The CSV row index restarts at 0 in every one of the five review files, which looks like a collision risk. Tested rather than assumed: because the files are split by product range, each product appears in exactly one file, so the pair collides **0 times** across all 1,094,411 rows. The key is sound.

7. Repository layout

```
CLAUDE.md                goals + running progress checkpoint (source of truth)
README.md                 mermaid architecture + ERD, run steps, verified numbers
requirements.txt
docker-compose.yml        Airflow 3.3.0, LocalExecutor
.gitignore / .env.example
explore.py                per-file exploration fns, toggled in main()
clean.py                  raw CSVs → data/processed/*.csv
ingest.py                  processed CSVs → sephora_oltp.raw via COPY
pipeline.py                local runner: --full-reload / incremental, timestamped logs
data/
  raw/                    (already present)      processed/
etl/
  extract.py              _full / _incremental / _all variants, get_watermark
  transform.py            build_dim_*, build_fact_reviews, FACT_COLUMNS, _drop_unmatched
  quality.py              DataQualityError, check_* fns, run_quality_checks
  load.py                 _records / _execute + ON CONFLICT DO NOTHING
  staging.py              STAGING_TABLES_SQL, stage/read/cleanup helpers
dags/
  sephora_dw_pipeline_staged.py
archive/
  sephora_dw_pipeline_v1_simple.py              earlier unstaged version, kept for comparison
sql/
  oltp/01_raw_schema.sql
  oltp/migrations/                3NF DDL → staging DDL → INSERT...SELECT loads → verify counts
  datawarehouse/migrations/       01_dim_product ... 10_alter_fact_add_unique
  analytics/                      dashboard-backing views
tests/ test_etl.py · test_quality.py
docs/   problem statement and data sources.md · 09_decision_log.md · checklist.md · screenshots/
logs/
```

8. Airflow DAG — `sephora_dw_pipeline_staged`

Same shape as the reference's staged DAG:

- `create_staging_tables` via `SQLExecuteQueryOperator`.
- **Four dimension branches in parallel** — `brand`, `product`, `customer`, `reviewer_profile` — generated from a `DIM_CONFIGS` list rather than hand-copied. Each is an `extract_<dim>_to_staging` → `load_<dim>_from_staging` pair scoped by `batch_id = run_id`, so a load failure retries without re-hitting the source and no row-level data crosses task boundaries via XCom. `product` waits on `brand` (FK dependency); the other two are fully independent.
- **Fact split into four staged tasks:** `extract` → `transform` → `quality` → `load`. The watermark (`MAX(submission_ts)` from `fact_reviews`) is captured in the **extract** task, before any write happens in the run, and travels as one small XCom dict. This is the exact race-condition fix the reference documents — re-querying the watermark mid-run would read a value that the same run had already advanced.
- The quality task raises `AirflowFailException` on a real `DataQualityError`, so bad data fails fast instead of burning the two-retry budget on something a retry can't fix.
- `cleanup_staging` with `trigger_rule="all_done"`, so staging rows are cleared whether the run succeeded or failed.
- `params={"full_reload": Param(False, type="boolean")}`, `schedule=None`, `catchup=False`, `retries=2`, `retry_delay=5min`.

`dim_customer` is the one heavy dimension (503,216 rows) — batched with `execute_values` rather than row-by-row `executemany`.

9. Analytics and dashboard

Four locked business questions, stated up front and never widened mid-build:

1. Which brands and categories earn the highest ratings, and which underperform?
2. How do review volume and average rating trend over time?
3. Does price predict satisfaction — do expensive products actually rate better?
4. Do reviewers with different skin types rate the same skincare products differently?

Logic lives in versioned SQL views under `sql/analytics/` — `vw_rating_by_brand`, `vw_rating_by_category`, `vw_review_trend_monthly`, `vw_rating_by_price_band`, `vw_rating_by_skin_type` — rather than being trapped inside the Power BI file.

Page 1 — Product & Brand Performance: KPI row (total reviews, average rating, recommendation rate, products covered), top and bottom brands by average rating with a minimum-review-count threshold, category treemap, price-band versus rating scatter.

Page 2 — Reviewer & Trend Insights: monthly review volume and rating trend, rating distribution, and a skin-type × rating matrix for Skincare — the view that the D2 fix makes trustworthy.

At least one DAX measure (recommendation rate with a review-count guard), live Postgres connection, screenshots committed to `docs/screenshots/`.

10. Build order

#	Stage	Output
---	-------	--------

1	Explore	<code>explore.py</code> ; findings written into the problem-statement doc (profiling above already done)
2	Clean	<code>clean.py</code> → <code>data/processed/</code> : dedup on the D4 key, type coercion, whitespace trim, <code>Grey</code> / <code>gray</code> and <code>notSureST</code> normalised, <code>highlights</code> and <code>ingredients</code> dropped, redundant review-side product columns dropped. Every rule logs rows in / out / dropped
3	OLTP	Raw schema, 3NF migrations, <code>ingest.py</code> (COPY), <code>INSERT...SELECT</code> staging loads, verify-counts migration
4	Warehouse	Star-schema migrations; <code>dim_date</code> seeded by <code>generate_series</code>
5	ETL	extract → transform → quality → load, then <code>pipeline.py</code> ; full, incremental, and re-run verified
6	Airflow	docker compose up, both Postgres connections registered, staged DAG run full → incremental → re-run, graph screenshotted
7	Analytics	Views, then Power BI, then screenshots
8	Docs	README with measured numbers, decision log filled in as decisions are made, checklist ticked

`CLAUDE.md` is updated at the end of every stage — it holds the goals, the locked questions, the state of each stage, and the measured row counts.

11. Deliberate deviations from the reference project

Deviation	Why
Explicit <code>3nf</code> schema between raw and staging	Course goal #3 requires a 3NF OLTP; the reference's OLTP was a source mirror only
Script-based ingestion (<code>ingest.py</code> , COPY)	The reference imported CSVs by hand in DBeaver — its own checklist flags that as a gap. 1.09M rows makes manual import impractical regardless
No bridge tables at all	The dataset's one genuine many-to-many (<code>highlights</code>) was evaluated and deliberately descoped — see D3
Junk dimension <code>dim_reviewer_profile</code>	The reference had no analog; four correlated low-cardinality attributes is the textbook case for one
<code>dim_customer</code> carries no descriptive attributes	See D2 — they aren't functionally dependent on the customer, and 13.69% of reviews would be mis-tagged

12. Verification plan

Every stage is verified by measurement, and the numbers go into the README — nothing is assumed.

Check	Expected result
Cleaning	Rows in / out / dropped logged per rule; exactly 1,040 duplicate reviews removed on the D4 key; <code>'Grey'</code> / <code>'gray'</code> collapsed to one value and <code>'notSureST'</code> mapped to Unknown
Ingestion reconciliation	raw → 3nf → staging counts reconcile, with every intentional drop explained
Referential integrity	0 orphan reviews after the 3NF load (0 in the source — the check proves the load introduced none)

Full load	<code>pipeline.py --full-reload</code> → ~ 1,044,880 fact rows (1,094,411 – 49,531 held back – 1,040 dupes), plus counts on all 5 dimensions
Incremental load	<code>pipeline.py</code> → ~ 49,531 new fact rows from 2023-01-01 onward
Idempotency, real case	Re-run full without resetting the watermark → full window re-extracted, 0 rows inserted
Idempotency, empty case	Re-run incremental with nothing past the watermark → 0 extracted, 0 inserted
Quality gate	<code>test_quality.py</code> feeds a null key and a negative count, asserts <code>DataQualityError</code> ; the gate logs pass/fail per table on every run
Airflow	All three run modes green end-to-end in the staged DAG, screenshotted
Dashboard	Each of the 4 locked questions answered by a named visual, cross-checked against the equivalent SQL

13. Environment status

Component	Status
Python	3.13.12 with pandas 3.0.3 — ready
PostgreSQL 16	Installed; service currently stopped — start before stage 3
Docker	29.5.2 — ready for the Airflow stack
Python packages to install	<code>psycopg2-binary</code> , <code>python-dotenv</code> , <code>sqlalchemy</code>
Power BI Desktop	Needed before stage 7, with the Npgsql provider for the Postgres connector